Agenda

- Redux

Redux

- redux is a javascript library which is used for global state management
- a state which will be shared with all the components of an application will be managed by the redux library
- used in the application where considerable data/information need to be shared with all components
- it can be used in any JS application (react/angular/vue/vanilla JS)
- alternatives to redux
    - flux, ngrx (angular), zustand

Redux toolkit

- a wrapper library provided to implement redux global store easily in different frameworks
- uses redux behind the scene
- concepts
    1. action
    - name of the reducer function which is used to invoke the reducer function
    2. reducer
    - function which is used to perform some operation on a state stored inside a slice
    3. store
    - collection of slices
    - used to store key-value pairs
- installation

```
npm install @reduxjs/toolkit
npm install react-redux
```

- create a slice
    - It is a collection of
    1. (global) state
        - collection of key vaue pair
    2. reducer
        - a function used to manage (read/update) the state
    3. actions(represents an event)
        - used to invoke a required reducer
        - similar to the reducer function name

SUNBEAM INFOTECH

```
// slices/counterSlice.js

import { createSlice } from '@reduxjs/toolkit'

// create a slice
const counterSlice = createSlice({
  name: 'counter',
  initialState: {
    counter: 0,
  },
  reducers: {
    incrementAction: (state) => {
      state.counter += 1
    },
    decrementAction: (state) => {
      state.counter -= 1
    },
  },
})

// export actions
export const { incrementAction, decrementAction } = counterSlice.actions

// export reducer
export default counterSlice.reducer
```

- create a store
- store is a collection of slices
- add a slice to store a counter to the global store
- slice is a part of store which is used to store a state member

```
// store.js

import { configureStore } from '@reduxjs/toolkit'
import counterReducer from './slices/counterSlice'

export const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
})
```

- add the store inside the application, so the application can access the store for storing the global state
- use Provider provided by react-redux to use the store for the entire application

```
// main.jsx

import { createRoot } from 'react-dom/client'
```

SUNBEAM INFOTECH

```
import App from './App.jsx'
import { Provider } from 'react-redux'
import { store } from './store'

createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <App />
  </Provider>
)
```

useSelector()

- used to read/select value from a slice (which is a part of store)
- returns a state of selected slice

```
import React from 'react'
import { useSelector } from 'react-redux'

function Counter1() {
  // useSelector will be used to get the value from counterSlice
  const { value } = useSelector((store) => store.counter)

  return (
    <div>
      <h3>Counter1</h3>
      <div>value = {value}</div>
    </div>
  )
}

export default Counter1
```

useDispatch()

- updating the value in a slice (which is a part of store)
- returns an object of dispatcher which dispatches the action to the selected slice
- once the slice receives the action, it executes the required reducer

```
import React from 'react'
import { useDispatch, useSelector } from 'react-redux'
import { incrementAction } from '../slices/counterSlice'

function Counter2() {
  // read the value from slice
  const { value } = useSelector((store) => store.counter)

  // get the dispatcher
  const dispatch = useDispatch()
```

SUNBEAM INFOTECH

```
  const onIncrement = () => {
    // sending the incrementAction
    dispatch(incrementAction())
  }

  return (
    <div>
      <h3>Counter2</h3>
      <div>counter = {value}</div>
      <button onClick={onIncrement}>increment</button>
    </div>
  )
}

export default Counter2
```

Redux Terminology

There's some important Redux terms that you'll need to be familiar

1. Actions
2. Reducers
3. Store
4. Dispatch
5. Selectors

1. Actions

- An action is a plain JavaScript object that has a type field. You can think of an action as an event that describes something that happened in the application. -The type field should be a string that gives this action a descriptive name, like "todos/todoAdded".
- We usually write that type string like "domain/eventName", where the first part is the feature or category that this action belongs to, and the second part is the specific thing that happened.
- An action object can have other fields with additional information about what happened. By convention, we put that information in a field called payload.
- A typical action object might look like this:

```
const addTodoAction = {
  type: 'todos/todoAdded',
  payload: 'Buy milk'
}
```

2. Reducers

- A reducer is a function that receives the current state and an action object, decides how to update the state if necessary, and returns the new state: (state, action) => newState.
- You can think of a reducer as an event listener which handles events based on the received action (event) type.

SUNBEAM INFOTECH

- Reducers must always follow some specific rules:

1. They should only calculate the new state value based on the state and action arguments
2. They are not allowed to modify the existing state. Instead, they must make immutable updates, by copying the existing state and making changes to the copied values.
3. They must not do any asynchronous logic, calculate random values, or cause other "side effects

- The logic inside reducer functions typically follows the same series of steps:

1. Check to see if the reducer cares about this action

- If so, make a copy of the state, update the copy with new values, and return it

2. Otherwise, return the existing state unchanged

- Here's a small example of a reducer, showing the steps that each reducer should follow:

```
const initialState = { value: 0 }

function counterReducer(state = initialState, action) {
  // Check to see if the reducer cares about this action
  if (action.type === 'counter/incremented') {
    // If so, make a copy of `state`
    return {
      ...state,
      // and update the copy with the new value
      value: state.value + 1
    }
  }
  // otherwise return the existing state unchanged
  return state
}
```

3. Store

- The current Redux application state lives in an object called the store.
- The store is created by passing in a reducer, and has a method called getState that returns the current state value:

```
import { configureStore } from '@reduxjs/toolkit'
const store = configureStore({ reducer: counterReducer })
console.log(store.getState())
// {value: 0}
```

4. Dispatch

- The Redux store has a method called dispatch.
- The only way to update the state is to call store.dispatch() and pass in an action object.

SUNBEAM INFOTECH

- The store will run its reducer function and save the new state value inside, and we can call getState() to retrieve the updated value:

```
store.dispatch({ type: 'counter/incremented' })
console.log(store.getState())
// {value: 1}
```

5. Selectors

- Selectors are functions that know how to extract specific pieces of information from a store state value.
- As an application grows bigger, this can help avoid repeating logic as different parts of the app need to read the same data:

```
const selectCounterValue = state => state.value
const currentValue = selectCounterValue(store.getState())
console.log(currentValue)
// 2
```

Redux Core Concepts document link

```
https://redux.js.org/tutorials/fundamentals/part-2-concepts-data-flow
```

SUNBEAM INFOTECH